VU2CPL DXCC Tracker

Reference document for the DXCC Tracker tab of the VU2CPL Node-RED shack automation. For the umbrella overview of all subsystems see [README.md](). For node IDs and operator notes see [CLAUDE.md]().

Overview

The DXCC Tracker monitors DX cluster spots in real-time against Club Log worked data and fires alerts for new DXCC entities, new bands, and new modes. All processing runs inside Node-RED on a Raspberry Pi — no external Python scripts required.

The flow connects to DX clusters via TCP, parses spots, resolves DXCC prefix via cty.xml, classifies against worked/confirmed data from Club Log, and sends alerts to the Node-RED dashboard, MQTT, FlexRadio, and Telegram.

Architecture

DX Cluster TCP → Login+Parse+Dedup → DXCC Prefix Lookup → Alert Classify → Dashboard + MQTT + FlexRadio + Telegram

- cty.xml downloaded and parsed inside Node-RED (HTTP + zlib decompression)
- Club Log API fetched directly via https lib (modes 0–3, `deleted=0`)
- Dashboard controls use `fetch()` + HTTP-in nodes — `send()` not used
- Filter state persists via localStorage (browser) + Node-RED file context store

---

Confirmed vs Unconfirmed Logic

Club Log matrix values (undocumented):

Value	Meaning
0	Not worked
1	Worked, no QSL
2	Confirmed — LoTW or paper QSL
3	Confirmed — eQSL only

The tracker uses `bands[mk] === 2` to classify confirmed slots. eQSL-only confirmations are intentionally excluded — matches ARRL DXCC award criteria.

The `parseMatrix` function uses an inverted initialisation pattern:

```
ew[adif][band] = ew[adif][band] || {confirmed:true, unconfirmed:true};
if (conf) ew[adif][band].confirmed   = false;  // value===2 clears confirmed flag
else      ew[adif][band].unconfirmed = false;  // others clear unconfirmed flag
```

Alert Types

Each spot can generate up to 2 independent alerts. NEW_DXCC is exclusive.

Alert	Colour	Condition
NEW DXCC	Red #f85149	Entity never worked
BAND	Blue #58a6ff	Band never worked
? BAND	Blue dim #2d6aad	Band worked but not LoTW/paper confirmed
MODE	Amber #e3b341	Mode never worked
? MODE	Amber dim #9a7030	Mode worked but not LoTW/paper confirmed

`DXCC Prefix Lookup + Alert Classify` processes all alert types independently and fires one `node.send()` per alert. A spot can generate both a BAND and MODE alert simultaneously.

Cluster Status Panel

Header bar above the main layout shows live status for all 4 DX cluster sources.

Source	Topic key	Host
VU2CPL	cluster3	vu2cpl.ddns.net:7550
VU2OY	cluster2	103.153.92.118:7550
VE7CC	cluster4	ve7cc.net:23
N2WQ	cluster1	cluster.n2wq.com:8300

Each card shows source name, spot count, and last spot time.

Colour	Condition
Green	Last spot within 5 minutes
Amber	Last spot 5–15 minutes ago
Red	No spot for >15 minutes or never seen

`Login + Parse + Dedup` updates `clusterStatus` flow variable on every valid spot (before dedup check). `Cluster Watchdog` inject (every 30s + on deploy) reads this and sends `{topic:'cluster_status'}` to the dashboard template.

HTTP Endpoints

Endpoint	Method	Function
`/dxcc/filters`	POST	Save band/type/mode/TTL filters
`/dxcc/refresh`	POST	Trigger Club Log re-fetch
`/dxcc/clear`	POST	Clear alert table

`/dxcc/blacklist-remove`	POST	Remove callsign (returns updated list)
`/dxcc/blacklist-add`	POST	Add callsign (returns updated list)
`/dxcc/stats`	GET	Return workedStats from flow context
`/dxcc/blacklist`	GET	Return blacklist array from file

All POST endpoints: 2-output pattern (output 1 → downstream, output 2 → http response). GET endpoints return JSON directly.

Dashboard Features

Feature	Detail
Cluster status header	4 source cards — name, spot count, last spot time
Band filters	160M–6M with All/HF/VHF presets
Alert types	DXCC / BAND / ? BAND / MODE / ? MODE
Mode filters	CW / Phone / Data
Spot lifetime	Slider 5–60 min
Block callsign	fetch POST /dxcc/blacklist-add
Unblock	Click tag → fetch POST /dxcc/blacklist-remove
Stats on load	GET /dxcc/stats on page init
Blacklist on load	GET /dxcc/blacklist on page init
Persistence	localStorage + Node-RED file context store

Credentials — Edit Once

Open the **Credentials** node and set:

```
var cfg = {
    cl_apikey   : 'YOUR_CLUBLOG_API_KEY',
    cl_email    : 'your@email.com',
    cl_password : 'your_clublog_password',
    cl_callsign : 'YOUR_CALLSIGN',
    tg_token    : 'YOUR_TELEGRAM_BOT_TOKEN',
    tg_chat_id  : 'YOUR_CHAT_ID'
};
// Projects user:
flow.set('cfg_flows_dir', os.homedir() + '/.node-red/projects/vu2cpl-shack');
// Non-projects user:
// flow.set('cfg_flows_dir', os.homedir() + '/.node-red');
```

Club Log Data — Persistence & Daily Refresh

`Fetch All Modes + Parse` saves to flow context RAM, file context store, and `nr_dxcc_seed.json` on every successful fetch.

Startup recovery sequence

1. Bootstrap checks file store (fastest, survives reboot)
2. File store empty → reads `nr_dxcc_seed.json` from `cfg_flows_dir`
3. Fallback: `~/.node-red/nr_dxcc_seed.json`
4. Club Log fetch fires at 12s → updates RAM and seed file

Failure handling

Scenario	Recovery
Club Log unreachable at startup	Retry at 90s
Both fetches fail	Bootstrap uses file store / seed file
Pi reboot	Bootstrap restores from file store instantly
Data stale	Daily 02:00 refresh

Startup Sequence

Step	Delay	Action
1	0.5s	Credentials → set cfg_* variables
2	2s	Bootstrap → file store OR nr_dxcc_seed.json
3	3s	Load blacklist
4	5s	Load cty.xml → ~340 entities
5	12s	Fetch Club Log → modes 0–3 → dxccReady=true
6	90s	Retry Club Log if step 5 failed
7	02:00	Daily re-fetch

Files on Pi

File	Purpose
`flows.json`	Main Node-RED flow (git tracked)
`clublog_dxcc_tracker_v7.json`	DXCC tab export (synced on every commit)
`nr_dxcc_seed.json`	Worked/confirmed seed
`nr_dxcc_blacklist.json`	Blocked callsigns

`DXCC.md`	This document
`DXCC_Tracker_README.pdf`	PDF version (always in sync)
`enable_file_context.sh`	One-time file context store setup

Standard Commit Sequence

```
cd ~/.node-red/projects/vu2cpl-shack
python3 -c 'import json; d=json.load(open("flows.json")); v=[n for n in d if
n.get("z")=="d110d176c0aad308" or n.get("id")=="d110d176c0aad308"];
json.dump(v,open("clublog_dxcc_tracker_v7.json","w"),indent=2);
print(len(v),"nodes")'
git add flows.json clublog_dxcc_tracker_v7.json DXCC.md DXCC_Tracker_README.pdf
git commit -m "v7: <description>"
git push
```

Technical Reference

Item	Value
MQTT broker	192.168.1.169:1883
FlexRadio	192.168.1.148:4992
Telegram bot	@nrdxccbot
Dedup window	60s per callsign + frequency
Club Log API	clublog.org/json_dxccchart.php
Repo	github.com/vu2cpl/vu2cpl-shack
Flow file	clublog_dxcc_tracker_v7.json

VU2CPL · Manoj · Bengaluru · MK83TE · Licensed 1993 · 9× BDXCC